

Project Overview

In App 3 we looked at interrupt latency in FreeRTOS on the ESP32-S3, this focused on the gap between "hardware interrupt fires" and how "application code actually responds." This project pairs a minimal ISR with two different bottom half wake mechanisms, then measures how the choice of mechanisms and the presence of competing load on Core 1 changes real, measured latency.

The core design follows the standard embedded pattern of keeping the ISR itself as thin as possible, then it debounces the button edge, pulses GPIO 19 as a hardware signal that you can see as a marker of "the ISR ran here," and then hands off to a bottom half rather than doing any real work. The scaffold implements two competing bottom half strategies, a binary semaphore given from the ISR and a direct-to-task notification, so the assignment becomes a controlled comparison rather than an isolated demo. Both paths report to per press latency in microseconds, letting the two mechanisms be judged on the same metric.

The layer on top of that is a load injection feature, which is a single `WITH_LOAD` compile time flag that brings in four periodic, priority ranked tasks that occupy Core 1 that compute work. Because the bottom half tasks sit at a fixed priority relative to the load tasks, toggling the flag creates a clean before and after scenario for observing priority driven preemption, specifically, how a higher-priority load task that can delay the wake path while lower priority ones cannot.

Finally, the project asks for a deliberately introduced fault which removes a yield from an ISR call, dropping `IRAM_ATTR`, or misusing a non ISR safe semaphore API to connect a specific RTOS rule to its concrete, observable failure mode. Instrumentation is done via Wokwi's logic analyzer against the two GPIO channels, converting scope level timing into the same microsecond latency figures the code reports at runtime, so the hardware trace and the software's self reported numbers can be cross checked against each other.